

Manual de Administración

Operaciones funcionales y de gobierno del Instituto Nacional de Educación Básica de Salcajá

SIGA-INEBI 1 de octubre de 2026

Contenido

Manual de Administración — SIGA-INEBI 3

1. Objetivo 3

2. Audiencia 3

3. Administración funcional vs. administración de infraestructura 3

4. Reglas previas 4

5. Cuentas y autorización 4

6. Configuración institucional y académica 9

7. Operación y trazabilidad 12

8. Escalamiento 15

9. Decisiones pendientes / contenido a confirmar antes de cerrar el manual final 17

Manual de Administración — SIGA-INEBI

1. Objetivo

Explicar, para personal administrativo autorizado, las operaciones funcionales y de gobierno que el sistema permite realizar: provisión y gestión de cuentas, asignación de roles y alcances, configuración institucional y académica, consulta de auditoría, gestión del almacenamiento documental y atención de incidencias administrativas. Este manual **no** es una guía para eludir controles de seguridad: toda operación sensible aquí descrita exige el permiso atómico correspondiente y queda registrada en bitácora.

2. Audiencia

Administración institucional (dirección, secretaría académica) y administradores funcionales autorizados del sistema. No está dirigido a personal de infraestructura (despliegue, base de datos, servidores): esas operaciones se cubren en el Manual Técnico (`02`) y en el Plan de Mantenimiento y Soporte (`08`).

3. Administración funcional vs. administración de infraestructura

Ámbito	Qué cubre	Dónde se documenta
Funcional (este manual)	Cuentas, roles, permisos, alcances, ciclos escolares, estructura institucional, auditoría, almacenamiento documental, incidencias de uso	Este documento
Infraestructura	Despliegue, variables de entorno, respaldo físico de base de datos y archivos, monitoreo de servidor, migraciones	`docs/manuals/02-technical-manual-instructions.md` , `docs/manuals/09-final-backup-and-recovery-plan-instructions.md`

Ningún procedimiento de este manual requiere acceso a servidor, consola de base de datos ni variables de entorno: todo se realiza desde la aplicación web con una cuenta que tenga los permisos atómicos indicados.

4. Reglas previas

- Todos los ejemplos usan datos ficticios (`admin.demo@siga.test`, "Instituto Demostración"). Nunca se documentan credenciales, tokens, cookies ni datos reales de estudiantes o personal.
- Cada pantalla o endpoint mostrado es el que realmente existe en el código actual; donde solo existe API y no hay pantalla dedicada, se dice explícitamente "disponible por API, sin pantalla propia todavía".
- Toda operación sensible indica el evento de auditoría (`apps.audit`) que queda registrado, consultable luego en la sección 7.

---

5. Cuentas y autorización

5.1 Modelo de autorización (para entender el resto de la sección)

SIGA-INEBI usa **RBAC con alcances obligatorios** (ADR-0005):

- Una cuenta (`UserAccount`) siempre está vinculada 1 a 1 con una **persona institucional** (`Person`); no existen cuentas huérfanas ni autorregistro público (RF-CTA-001, RF-CTA-002).
- Un **permiso atómico** (ej. `account.create`, `student.view_sensitive`, `grade.correct`) solo autoriza una operación si además viene acompañado de un **alcance** (institución, ciclo, grado, sección, materia, asignación docente, estudiante puntual, o un "módulo" completo como `module_key=students`). Un permiso sin alcance vigente se evalúa como si no existiera (RF-ALC-001).
- Un **rol** agrupa permisos atómicos; una cuenta puede tener **varios roles** vigentes a la vez y su autorización efectiva es la unión de todos ellos (RF-PER-002, RF-PER-003).

- **Denegación por defecto**: sin permiso y alcance explícitos para la operación exacta que se intenta, el sistema deniega y audita el intento (RF-PER-004, RF-PER-005). No hay "modo administrador" que ignore esta regla, salvo el superusuario técnico reservado para roles del sistema (ver 5.5).
- Los cambios de autorización (asignar/revocar rol, cambiar permisos de un rol) tienen efecto inmediato: se evalúan en cada operación, no hay sesión que deba cerrarse para que apliquen (RF-PER-006).

5.2 Catálogo de permisos atómicos

Consulta: `GET /api/identity/permissions/` — permiso requerido: `role.assign`. Sin pantalla dedicada todavía; se usa como catálogo de referencia al crear o editar roles. Devuelve el catálogo cerrado definido en `apps/identity/atomic_permissions.py` (p. ej. `account.create`, `account.disable`, `role.assign`, `scope.assign`, `student.view_sensitive`, `grade.correct`, `document.read`, `audit.read`, `academic_cycle.reopen`, entre otros). No se pueden inventar permisos fuera de este catálogo.

5.3 Crear y activar una cuenta

Paso	Detalle
Permiso requerido	`account.create`, con alcance (p. ej. `module_key=identity`)
Precondición	La persona institucional ya existe en el registro de personas y no tiene cuenta previa; si la cuenta es de encargado, la persona ya debe estar registrada como encargado activo
Acción	Provisión con desafío de activación: se crea la cuenta en estado `pending` (inactiva) y se emite un código de activación de un solo uso, con vigencia limitada y número máximo de intentos
Resultado esperado	Cuenta `pending`; el código se entrega por el canal institucional definido (fuera del sistema); el titular lo usa junto con su nueva contraseña para activar la cuenta, que pasa a `active`

Paso	Detalle
Evento de auditoría	`identity.account.created`, luego `identity.activation_challenge.issued`; al activar, `identity.account.activated`
Dónde	Pantalla de Cuentas (`AccountsPage`) y `POST /api/identity/accounts/`

Reglas duras: nadie puede reemitir su propio código de activación (auto-escalamiento prohibido), y la contraseña elegida en la activación pasa por la política de contraseñas configurada (longitud mínima, rechazo de contraseñas comunes).

5.4 Desactivar una cuenta

Permiso requerido	**`account.disable`, con alcance**
Precondición	La cuenta objetivo no es la propia cuenta del actor (prohibido autodesactivarse)
Advertencia previa	Si la persona tiene asignaciones docentes vigentes en un ciclo activo, el sistema devuelve una advertencia en lugar de desactivar de inmediato; el administrador debe confirmar explícitamente (`force=true`) para continuar
Resultado esperado	Cuenta pasa a `disabled`, `is_active=False`; conserva todo su historial (no se elimina a la persona ni sus registros)
Evento de auditoría	`identity.account.disabled`, con motivo declarado y detalle de las dependencias que existían
Regla de protección	No se puede desactivar a la última cuenta con permiso `account.create` activo: siempre debe quedar al menos un administrador capaz de dar de alta cuentas

5.5 Roles y permisos

Permiso requerido	**`role.assign` para crear/editar roles y para asignar/revocar roles a cuentas**
Precondición	Los permisos indicados deben pertenecer al catálogo atómico cerrado
Protección especial	Los **roles del sistema** (`is_system=True`) solo pueden modificarse con una cuenta de superusuario técnico; esto evita que un administrador funcional degrade el rol base que sostiene la operación
Asignación de rol	Exige alcance explícito (institución, ciclo, grado, sección, materia, asignación docente, estudiante o módulo); una asignación sin alcance es rechazada
Prohibición de autoescalamiento	Nadie puede asignarse o revocarse un rol a sí mismo; el intento se deniega y se audita
Resultado esperado	`RoleAssignment` creado con vigencia (`starts_at` / `ends_at`); si se declara alcance, se crea también el `ScopeGrant` correspondiente
Evento de auditoría	`identity.role.created` / `identity.role.updated` ; `identity.role_assignment.created` / `identity.role_assignment.revoked`
Dónde	API `POST/GET /api/identity/roles/` , `POST /api/identity/accounts/<id>/role-assignments/` , `DELETE /api/identity/role-assignments/<id>/` . **Sin pantalla propia todavía**: la gestión de roles y alcances se realiza hoy por API autorizada, no desde una pantalla dedicada del frontend

Revocar la última asignación con permiso `account.create` también está bloqueado por la misma regla de "nunca dejar el sistema sin administrador".

5.6 Sesiones y bloqueo por intentos fallidos

- **Bloqueo temporal por intentos fallidos** (RF-AUT-002): tras el número configurado de intentos de inicio de sesión fallidos consecutivos, la cuenta se bloquea temporalmente; se levanta automáticamente al expirar la ventana, o con un inicio de sesión exitoso posterior. Evento: `identity.login.denied` con motivo `temporarily_locked`.
- **Cierre administrativo de sesiones**: permiso `account.disable`. Cierra todas las sesiones activas de una cuenta (por ejemplo ante sospecha de acceso indebido) sin desactivar la cuenta. Evento: `identity.session.closed_administratively`. Disponible en la pantalla de Cuentas.
- **Duración de sesión por rol**: cada rol declara sus minutos de inactividad permitidos; si una cuenta tiene varios roles, se usa el más amplio para no interrumpir trabajo operativo en curso.

5.7 Restablecimiento asistido de contraseña

Permiso requerido	**`account.disable` (la misma autorización que gestiona la cuenta)**
Precondición	Cuenta existente; el restablecimiento lo inicia un administrador, no el propio titular por autoservicio público
Acción	Se emite un secreto de un solo uso con vigencia de 30 minutos; cualquier solicitud previa sin consumir queda revocada
Resultado esperado	Al consumir el enlace con una contraseña nueva válida, se cierran todas las sesiones previas de la cuenta
Evento de auditoría	`identity.password_reset.issued` y, al consumirse, `identity.password_reset.consumed`

5.8 Cambio de contraseña por el titular

Operación de autoservicio (no administrativa), incluida aquí porque un administrador puede necesitar explicarla a un usuario: exige la contraseña vigente, valida la política de contraseñas, cierra las demás sesiones activas y audita el cambio (`identity.password.changed`).

---

6. Configuración institucional y académica

> Toda operación de esta sección exige el permiso atómico `scope.assign` con alcance de institución (`require_assignment_scope()`), salvo la reapertura de ciclo, que usa el permiso dedicado `academic_cycle.reopen` . Esta autorización se añadió junto con este manual (issue #533): antes, estas escrituras solo exigían sesión autenticada, sin distinguir el rol administrativo.

6.1 Ciclos escolares

El ciclo escolar tiene tres estados: **en preparación** (`draft`), **activo** (`active`) y **cerrado** (`closed`). Solo puede existir un ciclo activo por institución a la vez. Mientras un ciclo está `closed` , las escrituras académicas sobre él quedan bloqueadas de forma transversal (matrícula, asignación docente, notas); solo la reapertura excepcional las vuelve a permitir.

Operación	Permiso / precondición	Resultado	Auditoría
Registrar ciclo	`scope.assign` sobre la institución; fechas válidas y sin solaparse con otro ciclo de la misma institución	Ciclo creado en `draft`	`academics.cycle.created`
Abrir (activar) ciclo	`scope.assign` ; el ciclo debe estar en `draft` ; no debe existir otro ciclo activo en la misma institución; la estructura de oferta (grados ofertados, secciones, jornadas, plan de estudios) debe estar completa — si falta algo, el sistema lista explícitamente qué falta y rechaza la activación	Ciclo pasa a `active`	`academics.cycle.activated`

Operación	Permiso / precondición	Resultado	Auditoría
Cerrar ciclo	`scope.assign`; el ciclo debe estar `active`; todas las unidades de evaluación deben estar cerradas y sin ventana de recuperación vigente	Ciclo pasa a `closed`; se **congelan** los resultados de cada matrícula activa (nota final por subárea y condición de promoción) como registro permanente, en la misma transacción	`academics.cycle.closed`
Reapertura excepcional	Permiso dedicado `academic_cycle.reopen` sobre la institución; exige un **motivo declarado** (`reason`)	El ciclo vuelve a `active`; sigue valiendo la regla de un solo ciclo activo por institución	Auditoría con el motivo registrado — operación excepcional, no una reversión rutinaria
Clonación hacia el ciclo siguiente	`scope.assign`; ciclo origen existente	Copia independiente de ofertas, jornadas, secciones, plan de estudios y, opcionalmente, asignaciones docentes, hacia un ciclo nuevo en preparación	Auditada; no mezcla datos entre ciclos

Dónde: pantalla de Ciclos (`CyclesPage` , `CycleFormWindow` , `CycleDetailWindow`).

6.2 Estructura institucional

Catálogos administrados con el mismo patrón: `scope.assign` sobre la institución para crear, actualizar o desactivar; creación validada y **baja lógica** (nunca eliminación física) cuando el registro ya tiene historia asociada.

- **Institución**: es una fila única que el sistema resuelve automáticamente; no hay una operación de alta/edición de institución en esta versión.
- **Sedes (Campus)**: código único, sede principal.
- **Niveles y grados**: catálogo ordenable (secuencia), con código autogenerado si no se indica.
- **Jornadas (Shift) y bloques de horario**: parámetros de jornada configurables por sede; los bloques no se superponen entre sí.
- **Aulas (Classroom)**: catálogo por sede con capacidad declarada; estados `disponible`, `inhabilitada` y `mantenimiento`. Un aula fuera de servicio bloquea **nuevas** asignaciones pero conserva las sesiones ya programadas (no se pierde historial). La capacidad declarada se usa solo como advertencia no bloqueante si una sección la supera.
- **Secciones**: cupo declarado por sección, con ocupación consultable.
- **Materias (Subject) y plan de estudios (CurriculumPlan)**: catálogo de materias por institución y su vínculo obligatorio/horas semanales por grado/nivel.
- **Sesiones de clase y publicación del horario**: registrar, clonar o desactivar una sesión, y publicar/despublicar el horario del ciclo (RF-HOR-009), exigen el mismo `scope.assign`.
- **Asignación docente (TeachingAssignment)**: vincula un docente a una sección/materia dentro de un ciclo, versionada en el tiempo (no se sobrescribe, se cierra una vigencia y se abre otra).

Todas estas operaciones de baja usan desactivación (`is_active=False`), nunca `DELETE` físico; las referencias históricas (por ejemplo, una sesión de clase ya dictada en un aula) quedan protegidas contra borrado accidental.

Dónde: pantallas `LevelsPage`, `CampusesPage`, `RoomsPage`, `SubjectsPage`, `TeachingAssignmentsPage` (módulo Académico).

6.3 Plantillas y configuración documental

Mismo permiso (`scope.assign` sobre la institución) para crear, actualizar o desactivar un tipo de documento o una plantilla.

- Catálogo de **tipos de documento** y de **plantillas**, con versiones inmutables (cada edición publicada queda archivada, no se sobrescribe la anterior) y una sola plantilla activa por tipo a la vez.
- El catálogo de **campos disponibles** para una plantilla es cerrado: no se puede insertar código ni marcadores arbitrarios, solo los campos dados de alta en el catálogo. Los campos sensibles están excluidos por defecto.
- Existe **vista previa** con datos de muestra antes de publicar una plantilla, sin que eso emita ni persista un documento real; la vista previa no exige `scope.assign` porque no modifica nada.

Dónde: pantallas `TemplatesPage`, `DocumentTypesPage`, `TemplateVersionsWindow`, `TemplatePreviewWindow`, `FieldTagsWindow`.

---

7. Operación y trazabilidad

7.1 Consulta de auditoría

Permiso requerido	`audit.read`
Qué se puede filtrar	actor, recurso, identificador de recurso, acción, rango de fechas
Resultado esperado	Lista filtrable de eventos; exportación (CSV) disponible con el mismo permiso
Garantía	Los eventos de auditoría son **inmutables**: no existe operación de edición ni borrado, ni siquiera masivo, sobre la bitácora (`AuditEvent` rechaza `update()` / `delete()` a nivel de modelo y de consulta)
Particularidad	La propia exportación de auditoría queda auditada (`audit.export.created`), y los intentos de acceso denegados a datos sensibles también generan un evento (`identity.authorization.denied`)

Permiso requerido	`audit.read`
Dónde	`GET /api/audit/events/`, `GET /api/audit/events/export/`. **Sin pantalla propia todavía**: la consulta de auditoría se hace hoy por API autorizada, no desde una pantalla dedicada

Cada asiento de auditoría registra quién (actor, con una copia de su nombre de usuario por si la cuenta se elimina más adelante), qué acción, sobre qué recurso, cuándo, y un contexto estructurado (antes/después, motivo declarado cuando aplica). Los valores sensibles (contraseñas, tokens, cookies) se excluyen automáticamente del contexto antes de guardarse.

7.2 Consumo de almacenamiento documental

Permiso requerido	**Cualquier cuenta autenticada (lectura informativa)**
Qué muestra	Total de bytes almacenados, cantidad de archivos, proyección de crecimiento por ciclo, y una **advertencia de umbral** cuando el consumo alcanza el límite configurado
Resultado esperado	Señal operativa para decidir ampliar almacenamiento o depurar adjuntos vencidos; **no purga ni elimina nada automáticamente**
Evento de auditoría	`documents.storage_consumption.read`

Existe además una verificación periódica de integridad de archivos almacenados (checksum), pensada para ejecutarse como tarea de mantenimiento, no como operación manual frecuente; ver Plan de Mantenimiento y Soporte (`08`) para la cadencia recomendada.

7.3 Alertas y umbrales de ausencia (reportes autorizados)

Permisos atómicos propios, verificados en `apps/reporting/api/views.py`:

Operación	Permiso requerido	Resultado	Auditoría
Configurar umbral de ausencia por jornada/ciclo	`reporting.absence_threshold.configure`	Se registra un umbral **nuevo**; el anterior se conserva (no se sobrescribe), para poder explicar retroactivamente por qué se emitió una alerta	Evento de servicio (`apps.reporting.services`)
Consultar tablero de alertas	`reporting.alert.view`	Lista de alertas, acotada además por el alcance de estudiante del actor (un actor con alcance de una sola sección no ve alertas de toda la institución)	—
Marcar alerta como atendida	`reporting.alert.acknowledge` **y** alcance vigente sobre el estudiante de esa alerta	Alerta marcada con responsable y fecha de atención	—
Recalcular alertas de una jornada	`reporting.alert.evaluate`	Reevalúa la asistencia de la fecha/jornada indicada y sincroniza las alertas resultantes	—

Dónde: pantallas `AlertsPage` , `AlertEvaluationWindow` , `AbsenceThresholdsWindow` .

7.4 Gestión de incidencias administrativas

SIGA-INEBI no tiene todavía un módulo interno de "tickets"; el registro oficial de incidentes, bugs y solicitudes de cambio son **GitHub Issues**, según define el Plan de Mantenimiento y Soporte (`docs/manuals/08-maintenance-and-support-plan-instructions.md`). Un administrador que detecta un problema debe:

1. Verificar primero si es un problema de autorización esperado (denegación por defecto) revisando el evento de auditoría correspondiente, antes de reportarlo como incidente.
2. Si es un incidente real, registrarlo en el repositorio con la severidad, impacto y evidencia (sin datos reales ni capturas con información sensible).
3. Dar seguimiento según la clasificación de severidad y los tiempos de respuesta definidos en el plan de soporte.

7.5 Conservación de la historia (regla transversal)

Por decisión de arquitectura (ADR-0006), **ningún dominio administrativo borra físicamente** un registro con historia: cuentas, roles, asignaciones, ciclos, aulas, secciones, documentos y matrículas se **desactivan, vencen o revocan**, nunca se eliminan. Esto es intencional y no es un defecto a corregir: permite reconstruir "quién tenía qué acceso cuándo" y "qué pasó con el expediente de un estudiante" en cualquier momento posterior.

---

8. Escalamiento

Situación	Qué hacer primero	Procedimiento de este manual	Plan relacionado
Acceso indebido sospechado (sesión comprometida, cuenta usada por otra persona)	Cerrar las sesiones activas de la cuenta de inmediato	§5.6 Cierre administrativo de sesiones; evaluar además desactivar la cuenta (§5.4) y revisar auditoría de esa cuenta (§7.1)	Plan de Mantenimiento y Soporte (severidad **Crítica**)
Cuenta bloqueada por intentos fallidos	Confirmar que es bloqueo temporal legítimo (ventana configurada) antes de intervenir	Si el titular no puede esperar la ventana, usar restablecimiento asistido (§5.7)	—
Pérdida de datos o inconsistencia detectada	No intentar corregir manualmente en la base de datos	Documentar el hallazgo con evidencia de auditoría; escalar como incidente	Plan de Mantenimiento y Soporte; si hay pérdida real de datos, Plan de Respaldo y Recuperación (`09`)

Situación	Qué hacer primero	Procedimiento de este manual	Plan relacionado
Datos inconsistentes entre pantallas/reportes	Revisar si hay un ciclo cerrado involucrado (los resultados quedan congelados al cierre)	Consultar auditoría del recurso afectado (§7.1) antes de reportar como bug	Plan de Mantenimiento y Soporte
Falla de una operación sensible (p. ej. cierre de ciclo rechazado, activación de cuenta fallida)	Leer el mensaje de precondición: casi siempre indica exactamente qué falta (unidad de evaluación abierta, estructura incompleta, etc.)	Resolver la precondición señalada antes de reintentar	Plan de Mantenimiento y Soporte, flujo de corrección

> Los Planes de Mantenimiento y Soporte (`08`) y de Respaldo y Recuperación (`09`) son entregables independientes de este manual; este documento enlaza a ellos en lugar de duplicar su contenido.

9. Decisiones pendientes / contenido a confirmar antes de cerrar el manual final

- **Verificar la suite de pruebas antes de fusionar** la autorización añadida a `apps/academics/api/views.py` (issue #533): este entorno no tenía PostgreSQL disponible para ejecutar `make test-backend`, así que los cambios se verificaron por lectura, colección de pruebas (`pytest --collect-only`, 1568 pruebas, sin errores) y `ruff check` / `format`, pero no con una corrida real. Ejecutarla es requisito antes de abrir el PR.
- **Cobertura de pruebas de rechazo**: se agregaron pruebas negativas (`..._rejects_without_assignment_scope`) para creación, activación y cierre de ciclo; el resto de los recursos de `scope.assign` (sedes, aulas, jornadas, niveles, grados, materias, secciones, plan de estudios, plantillas) solo se migraron a `admin_client` para seguir pasando, sin una prueba explícita de rechazo por recurso. Agregarlas completa el criterio de cierre del issue #533.
- **Canal de entrega del código de activación y del enlace de restablecimiento** de contraseña (correo institucional, impreso, otro): el código/token se emite por el backend, pero el canal de entrega al titular no está definido en el código ni en los requisitos revisados. Requiere decisión explícita de Dirección antes de documentarlo como procedimiento operativo.
- **Pantallas pendientes**: gestión de roles/permisos/alcances y consulta de auditoría hoy solo existen como API autorizada, sin pantalla dedicada. El manual final debe decir "disponible por API" o esperar a que exista pantalla, según lo que se entregue — no se debe prometer una pantalla que no exista en la versión entregada.
- **RTO/RPO y simulacro de recuperación** (Plan `09`) no están definidos todavía (ver `RNF-RES-002` en el catálogo de requisitos); el enlace de la sección 8 debe actualizarse cuando ese plan quede aprobado.
- Confirmar con el responsable de RF-CTA-003 si el flujo de activación por código de un solo uso ya descrito en §5.3 es el flujo final o sigue en ajuste, ya que el catálogo de trazabilidad (`docs/requirements/traceability-matrix.md`) y la tabla de requisitos tienen entradas duplicadas/inconsistentes para ese ítem que conviene depurar antes de publicar el manual.